

Introduction to Programming with Haskell

Functions and types

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Haskell

- A **functional programming language**

Haskell

- A **functional programming language**
- Programs are (mostly) just function definitions

Haskell

- A **functional programming language**
- Programs are (mostly) just function definitions
- Direct and succinct transcription of mathematical definitions

Haskell

- A **functional programming language**
- Programs are (mostly) just function definitions
- Direct and succinct transcription of mathematical definitions
- No need of an elaborate machine model to understand computation

Haskell

- A **functional programming language**
- Programs are (mostly) just function definitions
- Direct and succinct transcription of mathematical definitions
- No need of an elaborate machine model to understand computation
- Computation is like simplifying mathematical expressions

Haskell

- A **functional programming language**
- Programs are (mostly) just function definitions
- Direct and succinct transcription of mathematical definitions
- No need of an elaborate machine model to understand computation
- Computation is like simplifying mathematical expressions
- Many features are part of more mainstream languages

What is functional programming?

- Best understood by contrasting with the **imperative** or **procedural** style

What is functional programming?

- Best understood by contrasting with the **imperative** or **procedural** style
- $\sum_{i=0}^n i = 0 + 1 + \dots + n$

What is functional programming?

- Best understood by contrasting with the **imperative** or **procedural** style
- $\sum_{i=0}^n i = 0 + 1 + \dots + n$
- **Imperative / procedural style**

```
int sumUpto (int n) {  
    int sum = 0;  
    for (int i = 0; i ≤ n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

What is functional programming?

- Best understood by contrasting with the **imperative** or **procedural** style
- $\sum_{i=0}^n i = 0 + 1 + \dots + n$
- **Imperative / procedural style**

```
int sumUpto (int n) {  
    int sum = 0;  
    for (int i = 0; i ≤ n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

- Describes a **process** to compute the sum

What is functional programming?

- Best understood by contrasting with the **imperative** or **procedural** style
- $\sum_{i=0}^n i = 0 + 1 + \dots + n$
- **Imperative / procedural style**

```
int sumUpto (int n) {  
    int sum = 0;  
    for (int i = 0; i ≤ n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

- Describes a **process** to compute the sum
- Repeated change of **state** – the value stored in the variable **sum**

What is functional programming?

- A **style** of programming in which the basic method of computation is applying functions to arguments

What is functional programming?

- A **style** of programming in which the basic method of computation is applying functions to arguments
- How do we elaborate the meaning of the $\dots$ in $0 + 1 + \dots + n$?

What is functional programming?

- A **style** of programming in which the basic method of computation is applying functions to arguments
- How do we elaborate the meaning of the $\dots$ in $0 + 1 + \dots + n$?
 - $\sum_{i=0}^0 i = 0$

What is functional programming?

- A **style** of programming in which the basic method of computation is applying functions to arguments
- How do we elaborate the meaning of the $\dots$ in $0 + 1 + \dots + n$?
 - $\sum_{i=0}^0 i = 0$
 - $\sum_{i=0}^n i = \left(\sum_{i=0}^{n-1} i\right) + n$ for $n > 0$

What is functional programming?

- A **style** of programming in which the basic method of computation is applying functions to arguments
- How do we elaborate the meaning of the $\dots$ in $0 + 1 + \dots + n$?
 - $\sum_{i=0}^0 i = 0$
 - $\sum_{i=0}^n i = \left(\sum_{i=0}^{n-1} i\right) + n$ for $n > 0$
- The above mathematical definition can be transcribed directly to Haskell

```
sumUpto 0 = 0
```

```
sumUpto n = sumUpto (n-1) + n
```

Course aims

- Algorithmic problem solving

Course aims

- Algorithmic problem solving
- Transcribing mathematical problem specifications into functional programs

Course aims

- Algorithmic problem solving
- Transcribing mathematical problem specifications into functional programs
- Common data structures and analysis

Course aims

- Algorithmic problem solving
- Transcribing mathematical problem specifications into functional programs
- Common data structures and analysis
- Structuring large programming tasks using some advanced Haskell features

Functions

- A **function**

Functions

- A **function**
 - Takes one or more arguments

Functions

- A **function**
 - Takes one or more arguments
 - Returns a single result

Functions

- A **function**
 - Takes one or more arguments
 - Returns a single result
- **Function definition**

Functions

- A **function**
 - Takes one or more arguments
 - Returns a single result
- **Function definition**
 - An equation with a function name, argument names, and a **body**

Functions

- A **function**
 - Takes one or more arguments
 - Returns a single result
- **Function definition**
 - An equation with a function name, argument names, and a **body**
- Example function

```
double x = x + x
```

Functions

- A **function**
 - Takes one or more arguments
 - Returns a single result
- **Function definition**
 - An equation with a function name, argument names, and a **body**
- Example function

```
double x = x + x
```

- **Applying functions:** Replace actual arguments in place of argument names

```
double 3 = 3 + 3 = 6
```

Computations

- A longer computation

```
double (double 2)
```

```
double (2 + 2)
```

```
double 4
```

```
4 + 4    =    8
```

Computations

- A longer computation

```
double (double 2)
double (2 + 2)
double 4
4 + 4    =    8
```

- Computations need not be unique

```
double (double 2)
double 2 + double 2
(2 + 2) + double 2
4 + double 2
4 + (2 + 2)    =    4 + 4    =    8
```

Haskell

- A programming language for describing functions

Haskell

- A programming language for describing functions
- A function description has two parts

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs
- **Definition** or **rule** for computing outputs from inputs

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs
- **Definition** or **rule** for computing outputs from inputs
- Example function

```
sqr :: Int → Int      -- Type specification  
sqr x = x * x         -- Computation rule
```

Basic types

- **Int** – (64-bit) integers

Basic types

- **Int** – (64-bit) integers
 - Operations: $+$, $-$, $*$, $/$ (Note: $/$ produces **Float**)

Basic types

- **Int** – (64-bit) integers
 - Operations: **+**, **-**, *****, **/** (Note: **/** produces **Float**)
 - Functions: **div**, **mod**

Basic types

- **Int** – (64-bit) integers
 - Operations: **+**, **-**, *****, **/** (Note: **/** produces **Float**)
 - Functions: **div**, **mod**
 - Bounded range: **minBound**, **maxBound**

Basic types

- **Int** – (64-bit) integers
 - Operations: **+**, **-**, *****, **/** (Note: **/** produces **Float**)
 - Functions: **div**, **mod**
 - Bounded range: **minBound**, **maxBound**
- **Float** – Floating point (“**real numbers**”)

Basic types

- **Int** – (64-bit) integers
 - Operations: `+`, `-`, `*`, `/` (Note: `/` produces **Float**)
 - Functions: **`div`**, **`mod`**
 - Bounded range: **`minBound`**, **`maxBound`**
- **Float** – Floating point (“**real numbers**”)
- **Char** – Characters: `'a'`, `'%'`, `'7'`, ...

Basic types

- **Int** – (64-bit) integers
 - Operations: `+`, `-`, `*`, `/` (Note: `/` produces **Float**)
 - Functions: **div**, **mod**
 - Bounded range: **minBound**, **maxBound**
- **Float** – Floating point (“**real numbers**”)
- **Char** – Characters: `'a'`, `'%'`, `'7'`, ...
- **Integer** – Unbounded integers

Basic types

- **Int** – (64-bit) integers
 - Operations: `+`, `-`, `*`, `/` (Note: `/` produces **Float**)
 - Functions: **div**, **mod**
 - Bounded range: **minBound**, **maxBound**
- **Float** – Floating point (“**real numbers**”)
- **Char** – Characters: `'a'`, `'%'`, `'7'`, ...
- **Integer** – Unbounded integers
- Limited only by memory resources

Basic types ...

- **Bool** – Booleans: **True** and **False**

Basic types ...

- **Bool** – Booleans: **True** and **False**
- Operations: **&&**, **||**, **not**, ...

Basic types ...

- **Bool** – Booleans: **True** and **False**
- Operations: **&&**, **||**, **not**, ...
- Relational operators to compare **Ints**, **Floats** etc.

Basic types ...

- **Bool** – Booleans: **True** and **False**
- Operations: **&&**, **||**, **not**, ...
- Relational operators to compare **Ints**, **Floats** etc.
- **=**, **/=**, **<**, **≤**, **>**, **≥**

Defining functions

- `xor` (Exclusive or)

Defining functions

- `xor` (Exclusive or)
- Input two values of type `Bool`

Defining functions

- `xor` (Exclusive or)
- Input two values of type `Bool`
- Check that exactly one of them is `True`

```
xor :: Bool → Bool → Bool      -- why?  
xor b1 b2 = (b1 && (not b2)) || ((not b1) && b2)
```

Defining functions

- `isOrdered`

Defining functions

- `isOrdered`
- Input three values of type `Int`

Defining functions

- `isOrdered`
- Input three values of type `Int`
- Check that the numbers are in order

```
isOrdered :: Int → Int → Int → Bool  
isOrdered x y z = (x ≤ y) && (y ≤ z)
```

Running Haskell programs

- Haskell interpreter `ghci`

Running Haskell programs

- Haskell interpreter `ghci`
 - Interactively call built-in functions

Running Haskell programs

- Haskell interpreter `ghci`
 - Interactively call built-in functions
 - Load user-defined Haskell code from a text file

Running Haskell programs

- Haskell interpreter `ghci`
 - Interactively call built-in functions
 - Load user-defined Haskell code from a text file
 - Similar to how Python works

Running Haskell programs

- Haskell interpreter `ghci`
 - Interactively call built-in functions
 - Load user-defined Haskell code from a text file
 - Similar to how Python works
- Available for macOS, Windows, Linux

Using `ghci`

- Create a text file (extension `.hs`) with your Haskell function definitions

Using `ghci`

- Create a text file (extension `.hs`) with your Haskell function definitions
- Run `ghci` at the command prompt

Using `ghci`

- Create a text file (extension `.hs`) with your Haskell function definitions
- Run `ghci` at the command prompt
- Load your Haskell code with

```
:load myfile.hs
```

Using `ghci`

- Create a text file (extension `.hs`) with your Haskell function definitions
- Run `ghci` at the command prompt
- Load your Haskell code with

```
:load myfile.hs
```

- Call functions interactively within `ghci`

Compiling Haskell programs

- Can also write **standalone** Haskell programs

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function
- Example program – **hello.hs**

```
main = putStrLn "Hello there! I'm Haskell"
```

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function
- Example program – **hello.hs**

```
main = putStrLn "Hello there! I'm Haskell"
```

- Compile such programs using **ghc** (the **Glasgow Haskell Compiler**)

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function
- Example program – **hello.hs**

```
main = putStrLn "Hello there! I'm Haskell"
```

- Compile such programs using **ghc** (the **Glasgow Haskell Compiler**)
- **ghc hello.hs** produces the executable **hello**

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function
- Example program – **hello.hs**

```
main = putStrLn "Hello there! I'm Haskell"
```

- Compile such programs using **ghc** (the **Glasgow Haskell Compiler**)
- **ghc hello.hs** produces the executable **hello**
- Run the executable by issuing **./hello** from the command line

Compiling Haskell programs

- Can also write **standalone** Haskell programs
- Standalone programs require a **main** function
- Example program – **hello.hs**

```
main = putStrLn "Hello there! I'm Haskell"
```

- Compile such programs using **ghc** (the **Glasgow Haskell Compiler**)
- **ghc hello.hs** produces the executable **hello**
- Run the executable by issuing **./hello** from the command line
- We will concentrate on **ghci** for most of the course

Resources

- **Official Haskell page**

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)
- Online book: **Haskell for CMI** (**Arjun Maneesh Agarwal**, **Ryan Hota**, **Shubh Sharma**)

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)
- Online book: **Haskell for CMI** (**Arjun Maneesh Agarwal**, **Ryan Hota**, **Shubh Sharma**)
- NPTEL course: **Introduction To Haskell Programming**

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)
- Online book: **Haskell for CMI** (**Arjun Maneesh Agarwal**, **Ryan Hota**, **Shubh Sharma**)
- NPTEL course: **Introduction To Haskell Programming**
- Online book: **Learn You a Haskell for Great Good!**

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)
- Online book: **Haskell for CMI** (**Arjun Maneesh Agarwal**, **Ryan Hota**, **Shubh Sharma**)
- NPTEL course: **Introduction To Haskell Programming**
- Online book: **Learn You a Haskell for Great Good!**
- **Haskell Programming: from first principles**

Resources

- **Official Haskell page**
- Programming with Haskell ([Graham Hutton](#))
- Introduction to Functional Programming using Haskell ([Richard Bird](#))
- Thinking Functionally with Haskell ([Richard Bird](#))
- Online book: **Haskell for CMI** ([Arjun Maneesh Agarwal](#), [Ryan Hota](#), [Shubh Sharma](#))
- NPTEL course: **Introduction To Haskell Programming**
- Online book: **Learn You a Haskell for Great Good!**
- **Haskell Programming: from first principles**
- YouTube playlist: **Haskell for Imperative Programmers**

Resources

- **Official Haskell page**
- Programming with Haskell (**Graham Hutton**)
- Introduction to Functional Programming using Haskell (**Richard Bird**)
- Thinking Functionally with Haskell (**Richard Bird**)
- Online book: **Haskell for CMI** (**Arjun Maneesh Agarwal**, **Ryan Hota**, **Shubh Sharma**)
- NPTEL course: **Introduction To Haskell Programming**
- Online book: **Learn You a Haskell for Great Good!**
- **Haskell Programming: from first principles**
- YouTube playlist: **Haskell for Imperative Programmers**
- Plenty of other resources!